

中山大学计算机学院本科生实验报告

(2024 学年第 1 学期)

课程名称: Data structures and algorithms

任课教师: 张子臻

年级	23 级	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇
电话	13421145433	Email	linhy69@mail2.sysu.deu.cn
开始日期	2024.10.11	完成日期	2024.10.11

第一题

1. 实验题目

明明的随机数

2. 实验目的

明明想在学校中请一些同学一起做一项问卷调查,为了实验的客观性,他先用计算机生成了 N 个 1 到 1000 之间的随机整数 ($N \leq 100$), 对于其中重复的数字, 只保留一个, 把其余相同的数去掉, 不同的数对应着不同的学生的学号。然后再把这些数从小到大排序, 按照排好的顺序去找同学做调查。请你协助明明完成“去重”与“排序”的工作。

3. 算法设计

1.排序利用 sort 函数 2.去重, 在排序后的序列里利用前后对比对重复的元素去重

4. 程序运行与测试

1.排序

```
sort(v1.begin(),v1.end());
```

2.去重

```
for(int i=0;i<v1.size()-1;)
{
    if(v1[i]==v1[i+1])
    {
```

```

        v1.erase(v1.begin()+i);
    }
    else
        i++;
}

```

5. 实验总结与心得

1.该题比较简单 2.主要解决排序和去重的操作即可

第二题

1. 实验题目

Inversion Number

2. 实验目的

Let $A(1), \dots, A(n)$ be a sequence of n numbers. If $i < j$ and $A(i) > A(j)$, then the pair (i, j) is called an inversion pair. The inversion number of a sequence is one common measure of its sortedness. Given the sequence A , calculate its inversion number.

3. 算法设计

1. 利用归并计算逆序对
2. 合并两个已排序的子数组，并计算逆序对

4. 程序运行与测试

```

1.// 归并排序并计算逆序对
long long mergeSortAndCount(vector<int>& arr, int left, int right) {
    long long invCount = 0;
    if (left < right) {

```

```

        int mid = left + (right - left) / 2;

        invCount += mergeSortAndCount(arr, left, mid);
        invCount += mergeSortAndCount(arr, mid + 1, right);
        invCount += mergeAndCount(arr, left, mid, right);
    }
    return invCount;
}

```

2.// 合并两个已排序的子数组，并计算逆序对

```

long long mergeAndCount(vector<int>& arr, int left, int mid, int right) {
    int i = left, j = mid + 1;
    long long invCount = 0;
    vector<int> temp;

    // 合并两个子数组
    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp.push_back(arr[i++]);
        } else {
            temp.push_back(arr[j++]);
            invCount += (mid - i + 1); // 计算逆序对
        }
    }

    while (i <= mid) temp.push_back(arr[i++]);
    while (j <= right) temp.push_back(arr[j++]);

    // 将合并后的结果拷贝回原数组
    for (int k = left; k <= right; k++) {
        arr[k] = temp[k - left];
    }

    return invCount;
}

```

5. 实验总结与心得

1.算法主要理解归并排序的原理，利用递归将问题化简 2.通过这一道实验题可以加深对归并算法的理解

3.在归并排序函数中需要搞清楚函数之间的调用关系和逻辑

第三题

1. 实验题目

Mergesort for List

2. 实验目的

Please use mergesort to sort a linked list of data.

The linked list is defined as follows.

```
struct linkedlist{  
  
    int data;  
  
    linkedlist *next;  
  
};
```

Please implement the following function.

```
#include "mergeSort.h"//sort the List by mergesort and return the head of itvoid mergesort(linkedlist *&head, int len){  
  
    //add your code here  
  
}
```

3. 算法设计

- 1.设计主要函数：利用归并排序对链表进行排序
- 2.设计针对链表需要的合并函数和分割函数

4. 程序运行与测试

1.归并排序函数

```
void mergesort(linkedlist *&head, int len) {  
    if (head == nullptr || head->next == nullptr) {  
        return; // 基础情况：如果链表为空或只有一个节点  
    }  
}
```

```
linkedlist *a;
linkedlist *b;

// 将链表分成 'a' 和 'b' 两部分
split(head, &a, &b);

// 递归排序每一部分
mergesort(a, len / 2);
mergesort(b, len - len / 2);

// 合并已排序的部分
head = merge(a, b);
}
```

5. 实验总结与心得

1.实验同正常的归并排序相比，难点在于对链表的处理 2.任然需要掌握对归并排序原理的使用 3.需要根据链表，而不是一般的数组，写出对应的对链表进行处理的函数